# MAC-Based Message Authentication

Using HMAC (Hash-based Message Authentication Code) in Python

## 1. Introduction

A Message Authentication Code (MAC) is a short piece of information used to verify the integrity and authenticity of a message. It ensures that a message has not been tampered with during transmission and that it originates from the expected sender who possesses the shared secret key.

HMAC (Hash-based Message Authentication Code) is a specific construction for calculating a MAC using a cryptographic hash function in combination with a secret key. This activity demonstrates HMAC-SHA256 — one of the most widely used and trusted MAC schemes in modern cryptographic protocols such as TLS, JWT, and API authentication.

## 2. Objectives

- Design and implement a sender script that generates an HMAC-SHA256 MAC for a given message.
- Design and implement a receiver script that verifies the MAC upon receiving the message.
- Demonstrate how MAC ensures data integrity by testing with both an original and a tampered message.
- Understand the security properties of HMAC including resistance to timing attacks via hmac.compare_digest().
- Analyze the difference between authenticated and unauthenticated messages.

## 3. Theoretical Background

**What is a MAC?**

A MAC is computed using a secret key K and a message M, producing a fixed-size tag T = MAC(K, M). The receiver, who also holds K, independently computes the MAC from the received message and compares it to the received tag. If they match, the message is authentic and unmodified.

**HMAC Construction**

HMAC is defined by the RFC 2104 standard and is computed as:

```
HMAC(K, M) = H((K XOR opad) || H((K XOR ipad) || M))


Where:

  H    = Underlying hash function (SHA-256 in this activity)
  K    = Secret key (padded/hashed to block size)
```

The double hashing with separate paddings prevents both length-extension attacks and related- key attacks, making HMAC highly secure.

## CODE:

```
# sender_ridham.py - MAC-Based Message Authentication (Sender Side)


import hmac

import hashlib

import json


# Shared secret key (must be kept confidential between sender and receiver)

SECRET_KEY = b"supersecretkey123"


def generate_mac(message: str, key: bytes) -> str:

    """Generate HMAC-SHA256 MAC for the given message."""

    mac = hmac.new(key, message.encode('utf-8'), hashlib.sha256)

    return mac.hexdigest()


def send_message(message: str) -> dict:

    """Simulate sending a message with its MAC."""
```

```
    mac_value = generate_mac(message, SECRET_KEY)
    payload = {
        "message": message,
        "mac": mac_value
    }
    print("[+] Sender Name       : Ridham")
    print("[+] Original Message  :", message)
    print("[+] Algorithm         : HMAC-SHA256")
    print("[+] Generated MAC     :", mac_value)
    return payload

if _name_ == "_main_":
    message = "Hello! This is a secure authenticated message from Ridham."
    payload = send_message(message)
    with open("payload_ridham.json", "w") as f:
        json.dump(payload, f)
    print("[✓] Payload saved to payload_ridham.json")


# receiver_ridham.py - MAC-Based Message Authentication (Receiver Side)

import hmac
import hashlib
import json

# Same shared secret key as sender
SECRET_KEY = b"supersecretkey123"
```

```
def verify_mac(message: str, received_mac: str, key: bytes) -> bool:
    """Verify the MAC of a received message."""
    expected_mac = hmac.new(key, message.encode('utf-8'), hashlib.sha256)
    expected_hex = expected_mac.hexdigest()
    # Use compare_digest to prevent timing attacks
    return hmac.compare_digest(expected_hex, received_mac)

def receive_message(payload: dict, tampered: bool = False):
    """Simulate receiving and verifying a message."""
    message = payload["message"]
    received_mac = payload["mac"]

    print("[+] Receiver Name     : Ridham")

    if tampered:
        message = message + " [TAMPERED]"

    is_valid = verify_mac(message, received_mac, SECRET_KEY)

    if is_valid:
        print("[✓] MAC VERIFICATION PASSED - Message is AUTHENTIC")
    else:
        print("[✗] MAC VERIFICATION FAILED - Message has been TAMPERED")

    return is_valid

if _name_ == "_main_":
    with open("payload_ridham.json", "r") as f:
```

```
    payload = json.load(f)

print("--- Test 1: Original message ---")
receive_message(payload, tampered=False)

print("--- Test 2: Tampered message ---")
receive_message(payload, tampered=True)
```

## output:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS        powershell

PS C:\Users\user\Desktop\p> python ALA3.py
[+] Sender Name        : Ridham
[+] Original Message   : Hello! This is a secure authenticated message from Ridham.
[+] Algorithm          : HMAC-SHA256
[+] Generated MAC      : 03abd9e59b53c03c42551eadd491b5cc6e4fde5729de29abf58526cef4ade734
[√] Payload saved to payload_ridham.json
--- Test 1: Original message ---
[+] Receiver Name      : Ridham
[√] MAC VERIFICATION PASSED - Message is AUTHENTIC
--- Test 2: Tampered message ---
[+] Receiver Name      : Ridham
[X] MAC VERIFICATION FAILED - Message has been TAMPERED
PS C:\Users\user\Desktop\p>
```

## 4. Analysis & Discussion

**How MAC Ensures Data Integrity**

In Test 1, the receiver successfully verified the MAC because the received message was identical to what the sender originally hashed. The HMAC computed by the receiver matched the tag transmitted by the sender — confirming that no modification occurred during transit.

In Test 2, the message was altered by appending "[TAMPERED]" before verification. Even this small modification caused the HMAC-SHA256 output to change completely — a fundamental property of cryptographic hash functions known as the avalanche effect. The verification therefore failed, correctly signaling tampering.

**Why HMAC is Preferred over Plain Hashing**

| **Aspect** | Plain Hash (e.g., SHA-256) |
|---|---|
| **Secret Key** | No key used |
| **Authentication** | Anyone can compute hash |
| **Length Extension** | Vulnerable |
| **Forgery Resistance** | None |
| **Use Case** | Checksums, fingerprints |

**Timing Attack Prevention**

The receiver uses hmac.compare_digest() instead of the == operator for MAC comparison. A naive == comparison returns early as soon as it finds a mismatch, leaking timing information that attackers can exploit to guess the MAC byte-by-byte. compare_digest() runs in constant time regardless of where the mismatch occurs, fully preventing timing side-channel attacks.

**Limitations & Enhancements**

Replay Attacks: An attacker who captures a valid (message, MAC) pair can retransmit it. Adding timestamps or sequence numbers to the message before computing the MAC mitigates this.

Key Distribution: The shared secret key must be securely exchanged beforehand. In practice, asymmetric cryptography (e.g., RSA or ECDH) is used for key establishment.

Key Rotation: Long-term use of the same key increases risk. Periodic key rotation and use of derived keys (e.g., via HKDF) is recommended.

Digital Signatures: For non-repudiation with third parties, asymmetric digital signatures (ECDSA, RSA-PSS) should be used instead of symmetric MACs.